Head-Gesture Game Controller

Testing Guide: one computer, two computers, and event-day checks

This guide walks you through testing the system step by step, from a quick health check to a full rehearsal with several laptops. Each test says what you need, what to do, and what you should see when it works.

Item	Value
Relay server (WebSocket)	`wss://gesture-relay-xbs9.onrender.com`
Relay in a browser	`https://gesture-relay-xbs9.onrender.com/status`
Code repository	`https://github.com/Gowreesh-VT/Game-Dev-Event`
Event token	<EVENT_TOKEN> - ask the organizers. Wherever this guide says <EVENT_TOKEN>, type the real token.

Contents

Section	What it covers
1. How the system works	The three programs and how they connect
2. One-time setup	Install Python and the dependencies on Mac, Windows or Linux
Test 0	Relay health check (browser only, 1 minute)
Test 1	One computer, no webcam, no real key presses
Test 2	One computer, real key presses (key viewer)
Test 3	One computer, full webcam gesture test
Test 4	Two computers: station on one, game on the other
Test 5	Several teams and stations at once
Test 6	Safety tests (stuck keys, crashes, network drops)
Test 7	Local fallback relay (when internet or Render is down)
Reference	Gestures, hotkeys, troubleshooting, sign-off checklist

1. How the system works

There are three programs. Every one of them connects **out** to the relay server on the internet, so it works even on college Wi-Fi that blocks laptops from talking to each other.

```
station.py  ----->  relay (Render, internet)  <-----  listener.py
(webcam laptop)                                       (team laptop with the game)
```

Program	Runs on	What it does
station.py	Organizer laptop with a webcam	Watches the player's face. Head tilt/turn, mouth and blinks become key events.
relay.py	Render (already deployed)	Passes key events from a station to the one team it controls.
listener.py	Each team's laptop	Receives key events and presses the real keys, so the game reacts.
tools/fake_station.py	Any laptop	Sends a scripted key sequence. Lets you test without a webcam.
tools/key_viewer.html	Any browser	Shows which keys are being pressed. Use it in place of a game while testing.

Keys used: Arrow Up, Down, Left, Right, Z and Space. Nothing else is ever pressed.

> **NOTE** The listener releases every key on its own if the connection drops, if the station lets go, or if nothing arrives for 2 seconds. Keys can never stay stuck.

2. One-time setup

Do this once on every computer you test with. Use **Python 3.10 or newer** (3.12+ recommended).

Step A: Install Python

System	How
Windows	Download from python.org. During install tick **"Add python.exe to PATH"**.
macOS	Download from python.org, or `brew install python`. Avoid the built-in Python 3.9 if you can.
Linux	`sudo apt install python3 python3-venv python3-pip`

Step B: Get the code

```
git clone https://github.com/Gowreesh-VT/Game-Dev-Event.git
cd Game-Dev-Event
```

No git? On the GitHub page click **Code > Download ZIP**, unzip it, and open a terminal inside the folder.

Step C: Create a virtual environment and install

macOS / Linux (Terminal):

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-listener.txt          # every computer
pip install -r requirements-station.txt           # only the webcam / station computer
```

Windows (PowerShell or Command Prompt):

```
py -m venv .venv
.venv\Scripts\activate
pip install -r requirements-listener.txt
pip install -r requirements-station.txt           # only the webcam / station computer
```

NOTE Every new terminal window needs the environment activated again (`source .venv/bin/activate` or `.venv\Scripts\activate`). Your prompt should start with `(.venv)`.

Step D (optional): Save the settings in config.json

Copy `config.example.json` to `config.json` and fill in the server, token and team. The programs then need fewer options:

```
{
  "server": "wss://gesture-relay-xbs9.onrender.com",
  "token": "<EVENT_TOKEN>",
  "team": "01",
  ...
}
```

The commands in this guide spell out `--server` and `--token` in full, so they work with or without config.json.

TEST 0 — Relay health check

Time: 1 minute | You need: any browser

1. Open `https://gesture-relay-xbs9.onrender.com/health` in a browser.
2. If it takes up to a minute the first time, that's normal: the free server was asleep and is waking up.
3. Open `https://gesture-relay-xbs9.onrender.com/status`.

PASS IF /health shows **ok**, and /status shows JSON like `{"teams": [...], "stations": [...]}`.

TEST 1 — One computer: connection test (no webcam, no key presses)

Time: 5 minutes | You need: 1 computer, listener requirements installed

This proves the computer can reach the relay and that key events arrive. **--test** mode prints keys but never presses them, so it's safe anywhere.

1. Open **terminal 1** (environment activated) and start a test listener for team 01:

```
python listener.py --team 01 --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN> --test
```

2. Wait for `CONNECTED (team 01)` and `Registered as team 01`.
3. Open **terminal 2** (environment activated) and send scripted keys to team 01:

```
python tools/fake_station.py --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN> --team 01
```

PASS IF Terminal 1 shows `Now controlled by station: fake-station`, then KEY lines for up, down, left, right, z, space and diagonals (e.g. `held: up, left`). At the end it shows `No station is controlling this team`.

TEST 2 — One computer: real key presses

Time: 10 minutes | You need: 1 computer, a browser

This proves the listener can press real keys on this computer. Instead of a game we use the key viewer page.

1. Stop the test listener (Ctrl+C) and start it **without** `--test`:

```
python listener.py --team 01 --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN>
```

2. **macOS only:** if it prints *"macOS has NOT given this terminal permission to press keys"*, it opens the settings page for you. Switch on your terminal app under **Privacy & Security > Accessibility**, quit the terminal with Cmd+Q, reopen it and run the listener again. (This is not the top-level "Accessibility" settings page.)
3. Open `tools/key_viewer.html` in a browser (double-click the file) and **click on the page** so it has keyboard focus.
4. In terminal 2, run the fake station again (same command as Test 1). Then click back on the key viewer page straight away.

PASS IF The arrows, Z and SPACE boxes on the key viewer light up and go dark in step with the KEY lines in the listener terminal.

NOTE Keys go to whichever window has focus. If you click a terminal while keys are being pressed, the terminal receives them instead of the page.

TEST 3 One computer: full webcam gesture test

Time: 15 minutes | You need: 1 computer with a webcam, station requirements installed

The station and listener both run on the same computer. Head gestures go up to the relay and come back down to this computer as key presses.

1. Keep the listener from Test 2 running (terminal 1, without `--test`).
2. In terminal 2 start the station:

```
python station.py --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN> --name station1
```

3. First run only: it downloads the face model (about 4 MB). macOS asks for camera permission; allow it. Wrong camera? Try `--camera 1`.
4. Click the video window and press **C**. Look straight at the screen, eyes open, mouth closed, until it says **Calibrated** (3 s countdown + 1 s).
5. Press **T**, type **01**, press **Enter**. The top bar shows `Team 01` and a 10:00 timer starts.
6. Click the key viewer page. Try each gesture (see the gesture table on the reference page):

Do this	Expected key	Check
Tilt head up, hold	Up held	Releases when you return to neutral
Tilt head down, hold	Down held	
Turn head left / right	Left / Right	Your left, not the camera's
Turn and tilt together	Diagonal, e.g. Up + Left	Both boxes lit
Open mouth	Z held	
Close both eyes for ~0.5 s	Space held	Normal blinking must NOT press Space
Move out of the camera view	All keys released	Station shows NO FACE

PASS IF Every gesture lights the matching key on the key viewer and the station overlay, keys release when you return to neutral, and the station shows 25 fps or more (top right).

NOTE If a direction triggers too easily or too hard, press + or - in the station window, or tune the values in config.json (see ORGANIZERS.md). Note down the pitch / yaw / mouth / blink numbers from the overlay bars if you report a problem.

TEST 4

Two computers: station on one, game on the other

Time: 20 minutes | You need: computer A with webcam, computer B with a game or the key viewer

This is how the event works. Computer A is the organizer station; computer B is a team laptop. They can be on **different networks** (college Wi-Fi, mobile hotspot, home internet). Both only need internet access.

Computer B (team laptop)	Computer A (station)
Setup step C with requirements-listener.txt	Setup step C with requirements-station.txt
Run the listener, for example as team 07: `python listener.py --team 07 --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN>`	Run the station: `python station.py --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN> --name station1`
Wait for CONNECTED (team 07)	Press C to calibrate the player
Open the game (or key viewer) and **click it**	Press T: team 07 should be in the online list. Type 07, Enter
Listener prints: Now controlled by station: station1	Top bar shows Team 07 and the timer
Watch the game react	Player makes the gestures

1. Optional first step: run computer B's listener with `--test` and send keys from computer A with `tools/fake_station.py --team 07`. That checks the network path before involving the webcam.
2. Play for a few minutes. Then press **R** on the station: the listener prints `RELEASE ALL` and `No station is controlling this team`.

PASS IF Gestures on computer A move the game on computer B with no noticeable delay, and pressing R or quitting the station (Q) releases all keys on B.

TEST 5

Several teams and stations at once

Time: 20 minutes | You need: 2+ listener computers, 2 stations (a fake station counts)

1. Start listeners for two teams on two computers: team **01** and team **02**.
2. Open `https://gesture-relay-xbs9.onrender.com/status`. Both teams are listed with `"online": true`.
3. Station 1 takes team 01 (T, 01, Enter). Send keys: **only** team 01's listener shows them.
4. From another computer, try to take team 01 as a second station:

```
python tools/fake_station.py --server wss://gesture-relay-xbs9.onrender.com --token <EVENT_TOKEN> --team 01 --name station2
```

5. It must be refused with `Team 01 is controlled by station1`.
6. Run the same command with `--team 02`: it works, and only team 02 receives keys.
7. On station 1, press **R**. Now station 2 can take team 01.
8. **Duplicate team ID:** start a second listener with the same team ID as one already running. The older one exits with *"another laptop connected with this team ID"*.

PASS IF Keys only ever reach the team that the station selected, one station per team, and /status always shows which station controls which team.

TEST 6 Safety tests (nothing may stay stuck)

Time: 15 minutes | You need: 1-2 computers, key viewer open

For each case, hold a gesture (or use a fake station script with `hold`, e.g. `--script "hold right" --stay 30`) and then cause the failure.

What you do	Expected result
Quit the station with Q while a key is held	Listener: RELEASE ALL right away
Close / kill the station program (Ctrl+C, or close the terminal)	Listener releases all keys within about 2 s ("no message for 2s")
Turn off Wi-Fi on the listener computer for 10 s, then back on	Keys released at once (DISCONNECTED). It reconnects by itself (Reconnecting in ...s, then CONNECTED)
Turn off Wi-Fi on the station computer for 10 s	Listener releases keys within 2 s. Station reconnects and takes the team back automatically
Press P (pause) on the station while holding a gesture	Keys released; nothing is sent until P is pressed again
Walk out of the camera view	All keys released; station shows NO FACE
Start a listener with a wrong token	It exits with "the relay rejected the token"

PASS IF In every case the key viewer ends with **Held: none**, and both programs recover without being restarted (except the wrong-token case).

TEST 7 Local fallback relay (internet or Render down)

Time: 20 minutes | You need: a phone hotspot or Wi-Fi router, 2+ computers

If the hosted relay is unreachable on event day, one organizer laptop runs the relay itself. All computers must be on a network where laptops can reach each other, such as a phone hotspot. College Wi-Fi will not work for this.

1. Connect all test computers to the same hotspot.
2. On the relay laptop (environment activated):

```
python relay.py --port 8765 --token <EVENT_TOKEN>
```

3. Find that laptop's IP address. macOS: `ipconfig getifaddr en0`, Windows: `ipconfig` (IPv4 Address), Linux: `hostname -I`. Allow incoming connections if the firewall asks.
4. From another computer open `http://<IP>:8765/health`. It should show **ok**.
5. Repeat Test 4, but use `--server ws://<IP>:8765` (**ws://**, not wss://) for both the listener and the station.

NOTE Test 4 passes the same way over the local relay. Phone hotspots usually allow about 10 devices; only the stations and the teams demoing right now need to join.

Reference

Gestures

Gesture	Key
Tilt head up / down	Up / Down
Turn head left / right (the player's left/right)	Left / Right
Tilt and turn together	Diagonals (e.g. Up + Left)
Open mouth	Z
Close both eyes a little longer than a normal blink (0.2 s+)	Space
Hold the gesture	Key stays held; neutral releases it

Station hotkeys (click the video window first)

Key	Action
C	Calibrate: 3 s countdown, look straight, eyes open, mouth closed
T	Pick a team: type the ID, Enter to confirm, Esc to cancel
P	Pause / resume (pausing releases all keys)
R	Release the current team so another station can take it
+ / -	Sensitivity up / down
M	Mirror the preview (display only)
Q	Quit (releases all keys and the team first)

Troubleshooting

Problem	Fix
KEY lines appear in the listener but the game/viewer doesn't react	Click the game window to focus it. macOS: switch on the terminal under Privacy & Security > Accessibility, then quit and reopen it. Windows: if the game runs as Administrator, run the terminal as Administrator too.
ModuleNotFoundError (pynput, cv2...)	The environment isn't activated. Run the activate command; the prompt must show (.venv).
macOS: pip fails building pyobjc-core	Use the latest requirements-listener.txt, or install Python 3.12+ and make a new .venv.
Stuck on Connecting / Reconnecting	Check the internet, open /health in a browser, wait up to a minute if the relay was asleep.
"the relay rejected the token"	Check the token for typos.
Station: cannot open camera	Allow camera access for the terminal app; try --camera 1.
Team not in the T list / shows (offline)	That team's listener isn't connected; check it shows CONNECTED.
"Team 07 is controlled by station2"	Another station has it; ask them to press R.
Linux: keys don't arrive	pynput needs an X11 session; pick "Ubuntu on Xorg" at login.
Antivirus blocks the listener	It's a plain Python file that simulates keys; allow it.

Sign-off checklist

Tick each test as it passes. Record who tested, on which computers, and any notes.

	Test	Computers / OS	Tester	Notes
☐	0. Relay health check			
☐	1. One computer, --test			
☐	2. One computer, real keys			
☐	3. One computer, webcam			
☐	4. Two computers			
☐	5. Several teams / stations			
☐	6. Safety tests			
☐	7. Local fallback relay			

Event-day checklist

	Item
☐	About 10 minutes before demos, open /status so the relay is awake
☐	Every station has run station.py once (face model downloaded) and calibrates
☐	Each team ran listener.py --test at the registration desk and saw CONNECTED
☐	Each team checked real key presses with the key viewer (Test 2)
☐	A fallback laptop has relay.py installed and the organizers know Test 7
☐	The event token is shared with teams, not published in the public repo